MTH 4300
Spring 2026

Programming Problem Set 2: Recursion

Directions: Download the template files provided on Brightspace. Submit your source code for each problem individually via Gradescope, in `.cpp` format.

Submission Reminder: Submitting for feedback is optional. If you submit by the deadline and want feedback beyond the autograder, indicate that in your submission. You may resubmit unlimited times.
Important: Passing all test cases does not guarantee correctness. The autograder tests limited inputs. Test additional cases yourself.
AI Usage: If you use AI, document what you asked and what you learned in your code comments. State which tool (ChatGPT, Claude, Copilot, etc.). Be skeptical. AI can be confidently wrong. Your prompts should never include the entire problem; use AI to get unstuck, not to replace your thinking. If you collaborate with peers, acknowledge their contributions.
Conversation Links: If you use AI at any point (for tutoring, debugging, or any other purpose), include a link to your conversation(s) as a comment at the top of your `.cpp` file.
Using AI to Debug: If stuck, prompt AI to tutor you rather than solve for you:

> *"My program should [EXPECTED BEHAVIOR] but I'm getting [ACTUAL BEHAVIOR]. Don't explain the fix, ask me short guiding questions to help me find it myself. Keep a back-and-forth conversation. I should do most of the thinking."*

If AI gives too much detail: *"That's too much. Just hint at the next step."*
Note: While learning, avoid IDE tools that autocomplete or generate code for you (e.g., Copilot, IntelliSense suggestions). Writing code yourself builds the understanding you'll need for exams.

Be sure to read the SPECIFICATIONS carefully! And write comments!

Specifications and Notes (for both parts):

- There should be no loops.
- There should be no global variables (i.e., no variables defined outside of all functions).
- You can only use the header `iostream`.
- You cannot use sequences, vectors, or any advanced data structure that can store an arbitrarily long collection of integers.
- No input prompts or additional output should be used.
- Use the provided templates for each part. You may write a `main()` function to test your code on your own, but comment it out before submitting to Gradescope.

Note: Each problem has test cases on Gradescope. All test cases are visible, so you can see exactly which cases your code passes or fails. Use this feedback to improve your solution. You may resubmit as many times as you like. Remember, these assignments are for practice; use them to prepare for exams!

The autograder will call your functions directly and check the return values. You do not need to handle any input or output – just implement the functions as specified.

MTH 4300

2a) **countValidBinaryWarmup.cpp**

A binary string is a string that contains only 0's and 1's (for example, 10010 is a binary string of length 5).

We call a binary string *1-separated* if no two 1's appear next to each other. In other words, there must be at least one 0 between any pair of 1's.

For example: 10010 is 1-separated, but 1**1**000 is not (because two 1's are adjacent).

Write a recursive function `oneSeparated` that counts how many 1-separated binary strings there are of a given length, where the string ends in a specific digit. The function should take two parameters: the length of the string, and the ending digit (0 or 1).

You may assume that the input length will be a positive integer and the ending digit will be either 0 or 1.

Hint: Think about building the string from right to left. If the last digit is 0, what could the previous digit be? What if the last digit is 1?

Below are examples of what the function `oneSeparated` should return for the given input parameters.

- `oneSeparated(1,1)` should return `1` since the binary string consisting of a single 1 is 1-separated.
- `oneSeparated(3,1)` should return `2`, since the binary strings of length 3 ending in 1 that are 1-separated are 001 and 101.
- `oneSeparated(3,0)` should return `3`, since the binary strings of length 3 ending in 0 that are 1-separated are 000, 010, and 100.

2b) **countValidBinary.cpp**

This problem generalizes Part 2a. Instead of requiring just one 0 between any two 1's, we now require at least k zeros between any two 1's.

We call a binary string k-*separated* if every pair of 1's in the string has at least k zeros between them. For example, with k = 2 and length 6: 100100 is 2-separated, but 100101 is not (the last two 1's have only one 0 between them).

Note: a string does not need to contain more than one 1 to be k-separated. For example, 100 has only one 1, so there is no pair of 1's to violate the rule. That makes 100 k-separated for any value of k.

Write a recursive function `kSeparated` that counts how many k-separated binary strings there are of a given length, where the string ends in a specific digit. The function should take three parameters: the value of k, the length n of the string, and the ending digit (0 or 1).

You may assume that the input length and k will be positive integers, and the ending digit will be either 0 or 1.

Hint: Think about how your approach from 2a changes when you need k *zeros (instead of just one) before each 1. If a string ends in 1, then the previous 1 would have needed to appear at least* k + 1 *positions earlier, to leave room for* k *zeros in between.*

Below are examples of what the function `kSeparated` should return for the given input parameters.

- For k = 2, n = 3, `kSeparated(2,3,0)` returns 3 and `kSeparated(2,3,1)` returns 1, since 000, 010, 100, and 001, are the binary strings of length 3 that are 2-separated.

- For k = 3, n = 3, `kSeparated(3,3,0)` returns 3 and `kSeparated(3,3,1)` returns 1, since the same strings of length 3 that are 2-separated are also 3-separated.
- For k = 3, n = 4, `kSeparated(3,4,0)` returns 4 and `kSeparated(3,4,1)` returns 1, since 0000, 0010, 0100, 1000, and 0001 are the binary strings of length 4 that are 3-separated.

2c - challenge) **countValidBinaryChallenge.cpp**

In Part 2a, your function used two parameters: the length and the ending digit. In this challenge, write a recursive function `oneSeparatedChallenge` that takes only one parameter – the length of the string – and returns the *total* number of 1-separated strings of that length (ending in either 0 or 1).

You may *not* call your `oneSeparated` function from 2a. Your solution must be a self-contained recursion that does not use a separate helper to split by ending digit.

This problem is more challenging than 2a and 2b. You may notice a pattern quickly, but the real challenge is understanding *why* it works. In your comments, explain the reasoning behind your recursive solution – not just what the pattern is, but why it holds. If you submit this part and it works correctly, follows the instructions, and includes comments explaining your reasoning – great job!

Hint: Look at the values the function should return for small inputs (lengths 1 through 5). Do you notice a pattern? Can you figure out why it holds?

For example,

- `oneSeparatedChallenge(1)` should return 2, since both strings of length 1 (0 or 1) are 1-separated
- `oneSeparatedChallenge(2)` should return 3 since 00, 01, and 10 are the 1-separated strings of length 2
- `oneSeparatedChallenge(3)` should return 5

MTH 4300